

Parallelized-K-means-Image-Segmentation

URL: <https://github.com/BLS1202/Parallelized-K-means-Image-Segmentation>

Summary:

We implemented K-means clustering image segmentation in CUDA on GPU and in OpenMP on the CPU and compared the performance of non-optimized versions with these two parallelized versions. Our implementation demonstrated significant speedups. We also parallelized the initialization step of K-means++ in CUDA, and implemented a CUDA based image renderer that can run CUDA and OpenMP implementations of K-means clustering segmentation on image for every frame.

Background:

Basic algorithm:

The following is the pseudo code for k-means-clustering:

```
Function KMeans(data_points, K):
    centroids = select_random_points(data_points, K)
    Loop for max_iterations:
        For each point in data_points:
            min_distance = infinity
            closest_centroid_index = -1
            For each index j from 1 to K:
                distance = calculate_distance(point, centroids[j])
                If distance < min_distance:
                    min_distance = distance
                    closest_centroid_index = j
            Assign point to cluster closest_centroid_index
        For each index j from 1 to K:
            sum_of_points_in_cluster = 0
            count_of_points_in_cluster = 0
            For each point assigned to cluster j:
                sum_of_points_in_cluster += point
                count_of_points_in_cluster += 1

            If count_of_points_in_cluster > 0:
                centroids[j] = sum_of_points_in_cluster / count_of_points_in_cluster
    Return the final centroids and cluster assignments
```

This project focuses on parallelizing this algorithm using different tools and approaches.

OpenMP:

Data structure: in OpenMP algorithm, we use a structure of points to represent each pixel, where it includes the r, b, g information and which cluster the pixel get assigned to. The overall algorithm involves taking in a picture (an array of pixels), and the first step is to assign each pixel to the nearest cluster (calculated by sums the squares of the rgb difference), and the second step is to update the clusters based on the pixels assigned to it (get the rgb means respectively of the pixels), and this will iterate to get the final output array of pixels.

Workload Breakdown: We find out the most expensive part will be the assign each pixel to cluster and calculate the update for each cluster, which requires iteration through all the pixels and clusters.

Therefore, we do parallelization around the assignment step and update step separately. For assignment step, we parallel the calculation for each pixel. For update step, we first parallel around pixels to calculate the sum for each cluster, then for the second for loop we parallel the clusters to get the actual update. However, we notice that the bottle neck would be calculating the cluster sums, since many pixels correspond to a single cluster so the atomic add will be expensive. Therefore, we also implemented reduction method with openmp, where each core first updates it's local variable and then at the end combines all results together.

CUDA:

Data structures: In the k-means-clustering, we focused on parallelizing the algorithm inside each iteration, which involves calculating the distances for each point to its assigned cluster, and reassigning pixel values of the centroids for the next iteration. The key data structure in the implementation is a "Point" that has r, g, b float values, and we store the image data in an Array of Structure (AoS). In the initialization phase, we create a vector of Point struct and the final output of the algorithm will also be a vector of Point struct to represent the new image.

Workload Breakdown: We believe that the most expensive parts of the algorithm are looping through each point and assigning the new values to the centroids. We focus on parallelizing the assignment step and update step separately. To parallelize the assignment step, we will launch a kernel that allows each thread in the GPU to loop through the centroids and find the one with the shortest relative distance. We calculate the distance by taking the difference for each rgb values and sum up the squares of those differences. Then each thread will assign the point to the new cluster. For the update step, we will perform a parallelized version of reduction to calculate the sum and counts in the cluster, then reassign then update the centroid in parallel.

Dependency: The main dependency in the algorithm is that the updating of centroids must come after the assignment of the clusters. Therefore, we decide to separately parallelize the assignment step and the update step to exploit maximum parallelism.

Approach:

OpenMP:

Language/machines: We use OpenMP for CPU version. Used GHC machine with 8 cores and tested on PSC machine with 128 cores.

Implementation and optimization:

We first start with the basic sequential version of k means algorithm we get, and we've done two versions of the OpenMP optimizations separately. The overall algorithm involves assignment step and update step, where the assignment step gets parallelized around the pixels in order to calculate the distance between each pixel and each cluster, and for update step, we first parallel around pixels to calculate the sum for each cluster, then we parallel around the clusters for the second cluster to get the actual update. Since the memory for the sums of each cluster is shared across all cores, and each cluster corresponds to many pixels, when doing add operations, we need to have critical sections for the sums array.

Therefore, the first version deals with this problem with atomic operations (`#pragma omp atomic`). In this way, there won't be any race conditions. However, the atomic operation is relatively inefficient, because only one thread is allowed to update for a certain time step. So, this would be the bottleneck for the algorithm.

Therefore, in our final version, we replaced the atomic operations with reduction operations. For the reduction approach, since it will treat each array as a private copy for each thread and do the accumulation individually, and then reduction (sum) all arrays together, all threads can do the calculation at the same time instead of allowing only one thread to do increment.

Mapping:

We represent the picture as an array of points, where each point is represented by r, g, b, and clusterId. So, when parallel around the pixels, we map each point to a core and do calculation. since there are many points, and the calculation workload for each point is roughly the same, so we used the default static schedule and let the iterations divided evenly across the threads, which would be better for workload balance.

CUDA:

Language/Machine: We used NVIDIA CUDA API for GPU programming. Used GHC machines containing NVIDIA GeForce RTX 2080 B GPUs to test our implementations.

Implementation:

1. Initialization (Host): In the initialization phase, the CPU create necessary data structures and malloc those on the GPU's global memory. It will select K random pixel index and initialize the vector storing centroids data with those and copy them to the GPU.
2. Iteration (Device): For a fixed number of iterations, two kernels are launched:
 - k_means_v2 (Assignment & Partial Sum): The main kernel where each thread is responsible for one pixel point. It will find the closest centroid for its pixel. To avoid constantly reading global memory, we copy the centroid data to the shared memory first. After getting all the pixels to be assigned, sum reduction will be applied to each block, and each block's first thread will atomically add its sum to the global memory
 - update centroids (Update): A small kernel with K threads is launched. Each thread calculates the new position for one centroid by averaging the global sum
3. Final Output (Device/Host): We will use generate_output_image kernel where each thread replaces its pixel's color with the color of its assigned centroid. Then copy to the global memory vector that stores all the image data.

Mapping problem:

Data structures: We have several global memories and shared memories to store necessary data during the computation. The approach is data-parallel, so each thread performs the same core task, but with different pieces of data:

- d_inputImage (all pixels) in global memory, which is the input image data. We used shared memory data_point to access pixel data for each block.
- d_centroids (centroids) in global memory, but we also have centroids_W shared memories in kernels. Each centroids_W will be independent shared memory for each block.
- Partial_sums, partial_counts are also shared memories to do sum reductions inside each block.

Changes to serial algorithm: We made an algorithm change for the update step to make it suitable for parallelism. When implementing the update step, the main problem is how to avoid frequent atomic adds to the global memory when calculating the sum. A serial algorithm in this step uses a single loop to sum up points for each cluster. We made the task to be a two-stage parallel reduction. The first stage is to let each thread block do local sums for all centroids. Then we use atomic add to the global memory in each thread block. The first stage becomes a parallel reduction sum problem, which can provide a lot of performance speedup in a large image.

Optimization steps: We first implement a straightforward parallel implementation that does not involve shared memory optimization and reduction method. This implementation provided us with a 100x speedup, and we believe that the iteration over points is where the speedup mostly comes from. Then we optimized memory access with shared memories, and implemented the two-stage reduction of sum, which provides a two times speedup over the first version. Then we combined the assignment

step with the local sum step together and reduced the number of uses of syncthreads operation, which further optimized the speedup. We also tried to implement a data structure of structure of array (SoA), and tried different shared memory structures, but these did not provide further improvements in performance.

Results:

For OpenMP, we tested on both GHC machine (8 threads) and PSC machine (128 threads). For GHC machine, we tested the speedup for 1, 2, 4, 8, 16 threads; and for PSC machine, we tested the speedup for 16, 32, 64, 128 threads. Both of these speedups are tested against the sequential version of k means algorithm. We also choose 3 pictures for testing: camera_man (for simple segmentation test); lego (for moderate color diversity test); and forest (for hard color diversity and noise).

Original Images:



Result Images:



Our first experiment tested the computation time on 8 clusters and 20 iterations, and the baseline is the sequential version of the code. Based on the results, it shows that as the number of threads increases, the speedup also increases. For the method with atomic operations, there would be a obvious bottle neck with writing to shared memory, and the speedup is limited by the critical section access. For the reduction version, since each thread has a private copy of the array, do the accumulation individually, and then reduction (sum) all arrays together, all threads can do the calculation at the same time, which reduces the influence of the bottleneck. Therefore, the speedup gets increased to more than 7 for 8 threads. And since the machine only has 8 cores, 16 threads won't achieve more speedup because while some threads are working other threads need to wait for

resources, and this will increase the overhead.

GHC machine (Camera man: simple segmentation)

Sequential:

Clusters (K): 8

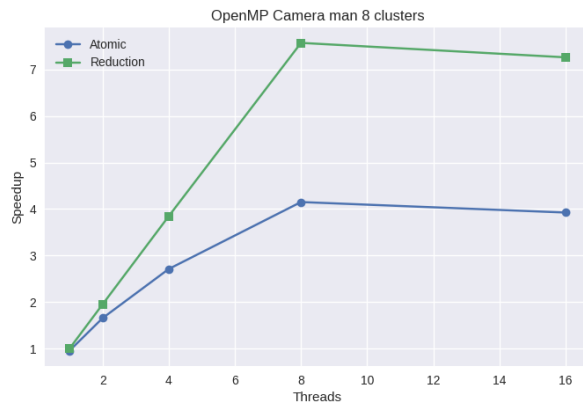
Max Iterations: 20

Computation time: 9.49

OpenMP:

Clusters (K): 8

Max Iterations: 20



#threads	1	2	4	8	16
With atomic	9.935 (x0.955)	5.735 (x1.655)	3.501(x2.711)	2.287(x4.151)	2.417(x3.925)
With reduction	9.480 (x1.001)	4.863 (x1.952)	2.467(x3.847)	1.253(x7.574)	1.307(x7.263)

GHC machine (lego: moderate color diversity)

Sequential:

Clusters (K): 8

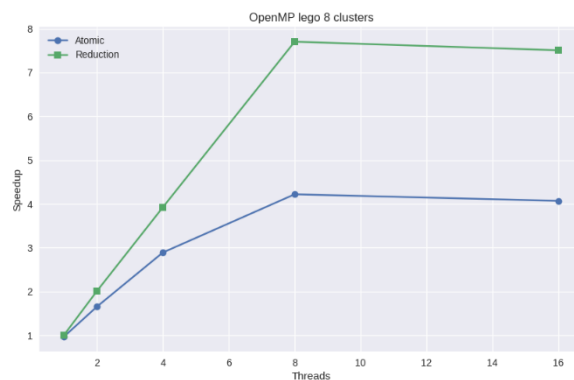
Max Iterations: 20

Computation time: 17.26

OpenMP:

Clusters (K): 8

Max Iterations: 20



#threads	1	2	4	8	16
With atomic	17.82 (x0.969)	10.43 (x1.654)	5.97 (x2.890)	4.09(x4.220)	4.24(x4.070)
With reduction	17.15 (x1.006)	8.59 (x2.010)	4.40 (3.923)	2.24(x7.705)	2.30(x7.509)

GHC machine (forest: hard color diversity)

Sequential:

Clusters (K): 8

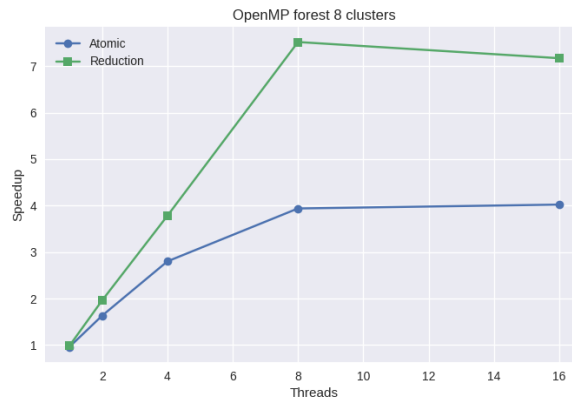
Max Iterations: 20

Computation time: 32.34

OpenMP:

Clusters (K): 8

Max Iterations: 20



#threads	1	2	4	8	16
With atomic	33.62 (x0.962)	19.79 (x1.633)	11.54 (x2.801)	8.21 (x3.939)	8.04 (x4.022)
With reduction	32.80 (x0.986)	16.49 (x1.961)	8.55 (x3.782)	4.30 (x7.519)	4.51 (x7.171)

And by comparing these 3 pictures, the camera man picture doesn't have many colors so it takes least amount of time to compute. The lego picture has many colors, and it takes longer time. For the forest picture, since it has many noises such as tiny color variations, and complex textures such as the trees and grasses, it takes longest time to compute.

Then we also tested the speedup by changing the cluster size (8, 16, 32, 64 clusters) for the camera_man picture. The speedup is also tested against the sequential version of k means.

GHC machine (Camera man: simple segmentation)

Sequential (Max Iterations: 20):

Clusters (K)	8	16	32	64
Computation time (sec)	9.49	18.668	37.09	73.15

OpenMP:

Clusters (K): 8

Max Iterations: 20

#threads	1	2	4	8	16
With atomic	9.935 (x0.955)	5.735 (x1.655)	3.501(x2.711)	2.287(x4.151)	2.417(x3.925)
With reduction	9.480 (x1.001)	4.863 (x1.952)	2.467(x3.847)	1.253(x7.574)	1.307(x7.263)

Clusters (K): 16

Max Iterations: 20

#threads	1	2	4	8	16
With atomic	19.343 (x0.965)	10.281 (x1.815)	5.658 (x3.298)	3.119 (x5.986)	3.355 (x5.562)
With reduction	18.711 (x0.997)	9.592 (x1.947)	4.927 (x3.789)	2.579 (x7.238)	2.534 (x7.364)

OpenMP:

Clusters (K): 32

Max Iterations: 20

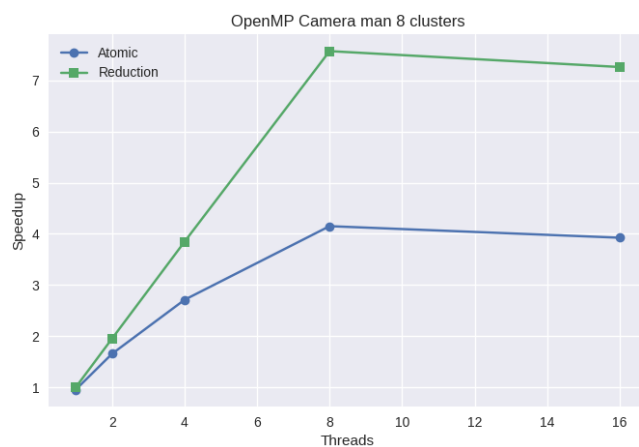
#threads	1	2	4	8	16
With atomic	38.118 (x0.973)	19.632 (x1.889)	10.351 (x3.583)	5.535 (x6.705)	5.738 (x6.467)
With reduction	38.019 (x0.975)	19.215 (x1.932)	9.754 (x3.803)	4.960 (x7.478)	5.163 (x7.183)

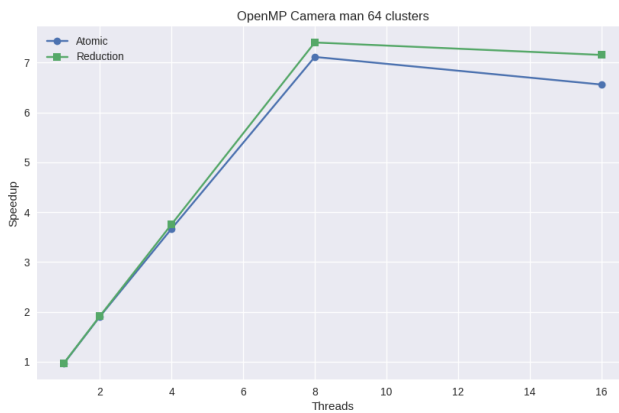
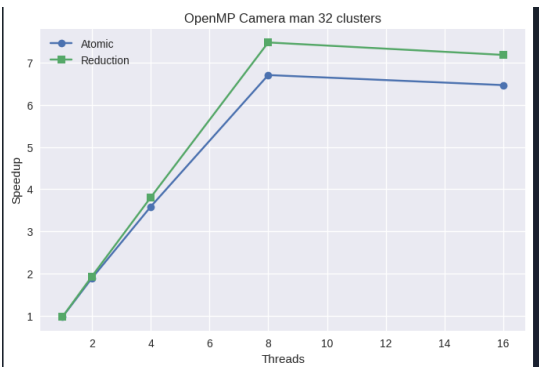
OpenMP:

Clusters (K): 64

Max Iterations: 20

#threads	1	2	4	8	16
With atomic	75.519 (x0.969)	38.387 (x1.906)	19.949 (x3.667)	10.277 (x7.116)	11.148 (x6.562)
With reduction	75.063 (x0.975)	38.241 (x1.914)	19.480 (x3.756)	9.876 (7.406)	10.223 (7.156)





Since it more clusters need more computation for each pixel (every pixel needs to calculate the distance to every cluster), the workload is larger, so it takes more time to compute. The speedup is roughly the same compared to 8 clusters, where 8 cores can get more than 7 times speedup for reduction method.

For PSC machines, we tested for 16, 32, 64, 128 threads, and the setup is 20 iterations and 64 clusters. The results show that as the number of threads increases to 64, the speedup also increases. However, as the number of threads increases from 64 to 128, the speedup does not have an obvious improvement. And we think that this is because the parallel resource is not fully used for 128 threads. Since the number of clusters is only 64, and there is a parallel loop that parallelizes around the number of clusters, when the number of threads is more than the number of clusters, some of the threads are

vacant and can't be fully used.

PSC machine (Camera man: simple segmentation)

Sequential:

Clusters (K): 64

Max Iterations: 20

Computation time: 73.15

OpenMP:

Clusters (K): 64

Max Iterations: 20

Compute Time (sec):

#threads	16	32	64	128
With atomic	3.708 (x19.73)	2.660 (x27.5)	1.567 (x58.46)	1.252 (x58.46)
With reduction	2.872 (x25.48)	1.464 (x49.98)	0.729 (x48.05)	1.522 (x48.05)

PSC machine (lego: moderate color diversity)

Sequential:

Clusters (K): 64

Max Iterations: 20

Computation time: 131.879

OpenMP:

Clusters (K): 64

Max Iterations: 20

Compute Time (sec):

#threads	16	32	64	128
With atomic	6.595 (x19.998)	4.612 (x.28.6)	2.735 (x48.22)	2.754 (x47.89)
With reduction	5.208 (x25.330)	2.545 (x51.860)	1.280 (x103.03)	0.937 (x139.8)

PSC machine (forest: hard color diversity)

Sequential:

Clusters (K): 64

Max Iterations: 20

Computation time: 249.550

OpenMP:

Clusters (K): 64

Max Iterations: 20

Compute Time (sec):

#threads	16	32	64	128
With atomic	13.013 (x19.18)	9.177 (x27.20)	5.627 (x44.33)	3.939 (x63.35)
With reduction	9.709 (x25.7)	4.931 (x50.61)	2.459 (x101.51)	2.821 (x88.45)

Deeper analysis: We also tested the initialization time and final updating image time for the program. We found out that we can further optimize the updating time by parallel around the pixels, which can help reduce the overall execution time for the algorithm.

For the Cuda version, we used GHC machines containing NVIDIA GeForce RTX 2080 B GPUs to test our implementations. We tested the speedup for different block sizes (X, Y dimension) on the 3 images described above, as well as different cluster sizes. For the cases with 8 clusters and 20 iterations:

GHC machine (Camera man: simple segmentation)

Cuda:

Clusters (K): 8

Max Iterations: 20

Blocksize	1*1	4*4	8*8	16*16	32*32
Basic	0.1090 (x87.06)	0.0804 (x118.03)	0.0789 (x120.30)	0.0769 (x123.40)	0.0790 (x120.13)
shared mem	0.208 (x45.63)	0.0138 (x687.68)	0.0134 (x708.21)	0.0184 (x515.76)	0.0354 (x268.18)

GHC machine (lego: moderate color diversity)

Cuda:

Clusters (K): 8

Max Iterations: 20

Blocksize	1*1	4*4	8*8	16*16	32*32
Basic	0.1845 (x93.55)	0.1321 (x130.62)	0.1246 (x138.46)	0.1218 (x141.74)	0.1230 (x140.33)
shared mem	0.333 (x51.81)	0.024 (x719.17)	0.021 (x821.9)	0.028 (x616.43)	0.048 (x359.58)

GHC machine (forest: hard color diversity)

Cuda:

Clusters (K): 8

Max Iterations: 20

Blocksize	1*1	4*4	8*8	16*16	32*32
Basic	0.3892 (x83.10)	0.2675 (x120.9)	0.2625 (x123.20)	0.2776 (x116.51)	0.2656 (x121.77)
shared mem	0.578 (x55.97)	0.051 (x643.12)	0.041 (x789.51)	0.054 (x598.89)	0.105 (x307)

Results show that there is a huge speedup for shared memory optimization, as the speedup can reach around 700 times compared to the sequential version. This is because shared memory reduces global memory access and can improve memory throughput. Also notice that blocksize can reach optimization at around 8*8/16*16 block size for most of the testcases. And we think that is this because for very large blocks, resource contention could reduce performance, and for very small blocks, there is not enough parallelism. And the similar results holds for all easy, moderate, and hard test cases.

For camera_man.jpg, 20 iterations, and (8, 16, 32 clusters):

CUDA:

Clusters (K): 8

Max Iterations: 20

Blocksize	1*1	4*4	8*8	16*16	32*32
Basic	0.1090 (x87.06)	0.0804 (x118.03)	0.0789 (x120.30)	0.0769 (x123.40)	0.0790 (x120.13)
shared mem	0.208 (x45.63)	0.0138 (x687.68)	0.0134 (x708.21)	0.0184 (x515.76)	0.0354 (x268.18)

Clusters (K): 16

Max Iterations: 20

Blocksize	1*1	4*4	8*8	16*16	32*32
shared mem	0.207 (x90.18)	0.0137 (x1362.04)	0.0132 (x1414.25)	0.0174 (x1072.87)	0.0308 (x605.49)

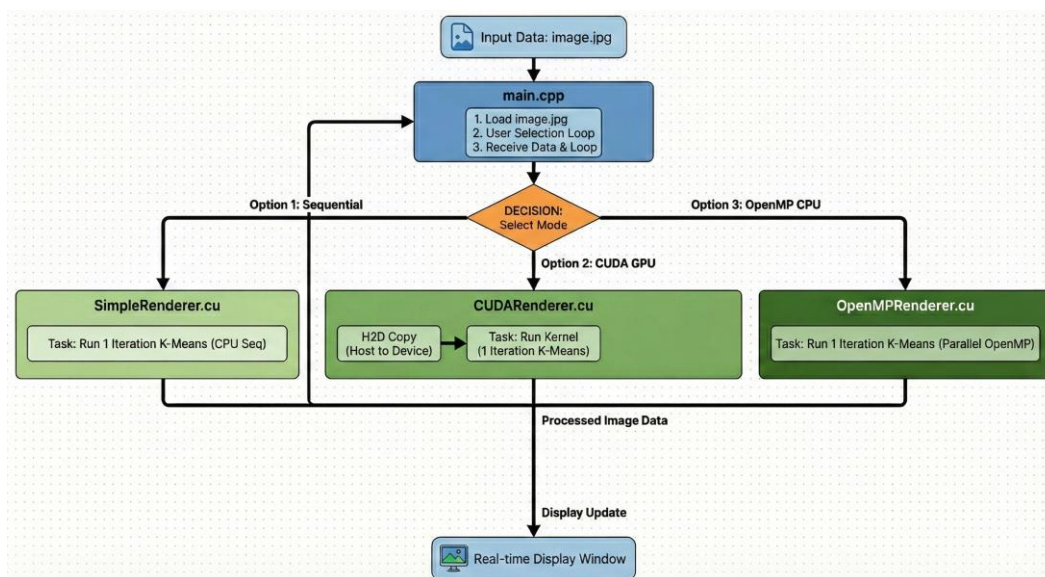
Clusters (K): 32

Max Iterations: 20

Blocksize	1*1	4*4	8*8	16*16	32*32
shared mem	0.2083 (x178.11)	0.0496 (x747.78)	0.0130 (x2853.08)	0.0168 (x2206.55)	0.0282 (x1315.6)

Based on the results, we found out that unlike the sequential version, increasing the cluster size affects little for the computation time for CUDA. We believe the reason is that CUDA runs thousands of threads in parallel and the distance calculation for multiple clusters is done in parallel for each thread, so when the number of clusters (K) increases moderately, this extra computation is tiny compared to the massive parallelism of the GPU. Also, since cluster centroids are stored in shared memory, which is extremely fast, accessing centroids for distance computation is very cheap, so adding more clusters does not linearly increase memory latency. Therefore, the highest speedup for 32 clusters can reach around 2800 speedup.

In order to demonstrate the output result of our k means image segmentation algorithm, we did an image Segmentation Process Renderer: To demonstrate the image segmentation process, we developed a renderer program in CUDA that can run different implementations of k-means clustering (Single thread sequential version, OpenMP, CUDA). The renderer can display the image for each iteration of update for the segmentation process.



(Workflow of the renderer, diagram generated by Gemini 3 Nano Banana)



(Running renderer with displayed segmented image)

Additional (K-means++ Implementation):

Background: In K-means clustering, initial centroids are usually initialized randomly, so there is certain probability in which the centroids are selected undesirably along the data points. K-means++ addresses this initialization issue by selecting one initial centroid, then select subsequent cluster centroids by choosing them from the remaining data points with probability proportional to the squared distance between points and the existing centroids.

The pseudo code below is the algorithm:

```
function kmeans++ (k, points) is
    // Initialize list of centroids with one randomly selected point
    centroids  $\leftarrow$  empty list
    firstIndex  $\leftarrow$  random integer from 0 to length(points) - 1
    centroids.append(points[firstIndex])

    // Choose remaining k - 1 centroids
    while length(centroids) < k do
        distancesSquared  $\leftarrow$  empty list

        // For each point, compute squared distance to nearest selected centroid
        for i  $\leftarrow$  0 to length(points) - 1 do
            point  $\leftarrow$  points[i]
            minDistance  $\leftarrow$  distance(point, centroids[0])
            for j  $\leftarrow$  1 to length(centroids) - 1 do
                d  $\leftarrow$  distance(point, centroids[j])
                if d < minDistance THEN
                    minDistance  $\leftarrow$  d
            distancesSquared.append(minDistance * minDistance)

        // Choose next centroid with probability proportional to  $D(x)^2$ 
        total  $\leftarrow$  sum of distancesSquared
        threshold  $\leftarrow$  random number from 0 to total
        cumulative  $\leftarrow$  0
        for i  $\leftarrow$  0 to length(points) - 1 do
            cumulative  $\leftarrow$  cumulative + distancesSquared[i]
            if cumulative >= threshold THEN
                centroids.append(points[i])
                break

    // Run k-means
    clusters  $\leftarrow$  kmeans(k, points, centroids)
    return clusters
```

(This pseudo code is from Wikipedia)

Approach for optimization:

We implemented the data-parallel version of the initialization step in CUDA. The sequential steps that we focused on for optimization are the two iterations over points. When calculating the squared

distance, we implemented a kernel that allows each thread to compute its squared distance relative to the existing centroids. For the step finding the point just above the distance threshold, partial sum reduction is used to parallelize the loop. We first allow each thread block to calculate partial sum, then use a vector to store these sums, and iterate over the vector until we reach above the threshold. After that, we launch one thread block to iterate over a range of pixels and find the new centroid.

Initialization time with K = 4:

	K_means++ sequential (sec)	K_means++ parallel (CUDA) (sec)
Time	0.516	0.082
Speedup	1.00x	6.29x

Initialization time with K = 8:

	K_means++ sequential (sec)	K_means++ parallel (CUDA) (sec)
Time	1.913	0.085
Speedup	1.00x	22.51x

Initialization time with K = 16:

	K_means++ sequential (sec)	K_means++ parallel (CUDA) (sec)
Time	7.63	0.084
Speedup	1.00x	90.83x

From the results, we see that our implementation of K-means++ provided enough speedup and has scalability relative to the number of clusters.

References:

- [1] Kartabyakrishna, “Kmeans-parallelization/K-means CUDA.cu at main · Kartabyakrishna/Kmeans-Parallelization,” GitHub, <https://github.com/kartabyakrishna/Kmeans-Parallelization/blob/main/K-Means%20CUDA.cu> (accessed Dec. 7, 2025).
- [2] “K-means++,” Wikipedia, <https://en.wikipedia.org/wiki/K-means%2B%2B> (accessed Dec. 7, 2025).
- [3] “Accelerating K-means with Cuda,” luigi cennini, <https://www.luigicennini.it/projects/cuda-kmeans/> (accessed Dec. 7, 2025).
- [4] Optimizing parallel reduction in CUDA Mark Harris Nvidia Developer Technology, https://developer.download.nvidia.com/compute/cuda/1.1-Beta/x86_website/projects/reduction/doc/reduction.pdf (accessed Dec. 8, 2025).
- [5] Detailed analysis and optimization of CUDA K-means algorithm | proceedings of the 49th International Conference on Parallel Processing, <https://dl.acm.org/doi/abs/10.1145/3404397.3404426> (accessed Dec. 8, 2025).